Inventory Image Tagging, Phase 2: A Small CNN, an Ablation Study and a CNN–Classifier Hybrid

Aryan, Divyanshu Singh, Jatin Kumar Singh, Jatin Verma, Kushagra Bhardwaj
Enrolments: 2401020013, 2401010161, 2401010199, 2401010200, 2401010247
Section B. Contact: aryan.kinha2024@nst.rishihood.edu.in

***Abstract*—Phase 1 classified Fashion-MNIST clothing pictures with fixed descriptions (pixels, HOG) and classical classifiers; the best of its eight core models, Pixels with Random Forest, reached 87.61% test accuracy and confused shirts, T-shirts, pullovers and coats. We train a small convolutional neural network (CNN) on the same 48,000 training pictures, choose every setting on the same 12,000 validation pictures, and score the official 10,000 test pictures once. An ablation study with three seeds per setting tests batch normalisation, dropout, augmentation and depth. We also test a hybrid that gives the CNN's 64 learned features to a Phase 1 classifier. The CNN reaches 91.98 ± 0.24% test accuracy and the hybrid 92.49 ± 0.19%. Shirt recall rises from 59.2% to 73.6% (CNN) and 75.7% (hybrid). Most ablation differences are close to the seed-to-seed spread, and dropout of 0.4 reduced accuracy.**

***Index Terms*—Fashion-MNIST, convolutional neural network, ablation, regularisation, hybrid model**

I. Introduction

Phase 1 described each 28×28 picture by a fixed list of numbers chosen before any model saw the data. Flattened pixels retain every value and position, but a vector classifier has no built-in local filters or weight sharing; HOG summarises local edge directions but fixes the description in advance. Its best model confused upper-body categories: only 59.2% of shirts were found.

A CNN learns its own description. Small shared filters measure local patterns, pooling keeps the strongest responses, and later layers combine them. We ask three questions. Does a small CNN beat the Phase 1 models on the same protocol? Which of its ingredients matter? Does a classical classifier on the CNN's learned features improve on the CNN's own last layer? The code, saved predictions and checkpoints are in the project repository.

II. Related Work

LeCun et al. [1] introduced convolution, weight sharing and pooling for document recognition. Krizhevsky et al. [2] showed that ReLU units, dropout and augmentation let a deep CNN generalise on large images. Srivastava et al. [3] describe dropout; Ioffe and Szegedy [4] describe batch normalisation and note it can also regularise. Kingma and Ba [5] describe Adam. Prechelt [6] compares early-stopping criteria. Simard et al. [7] found that distorting training pictures slightly helps a CNN.

For the hybrid, Razavian et al. [8] fit a linear SVM on features from a pretrained CNN and obtained strong results; Tang [9] replaced the softmax layer with a linear SVM trained jointly. We differ from both: our CNN is trained on this dataset only, and the classifier is fitted afterwards. Xiao et al. [10] introduced Fashion-MNIST. Reported gains from batch normalisation, dropout and augmentation on small CNNs are modest, so we measure them with several seeds instead of assuming them.

III. Data and Protocol

We reuse the Phase 1 split: official training pictures divided by a stratified split with seed 42 into 48,000 for training and 12,000 for validation, plus the official 10,000 test pictures. Pixels are divided by 255. All training, selection and early stopping use training and validation pictures only. The settings were written to *reports/phase2/frozen.json* before the test pictures were scored, and a marker file blocks a second test run. Phase 1 predictions on the same test pictures give a paired comparison.

Every setting was trained with three seeds (42, 1, 2); we report the mean and sample standard deviation. Macro F1 is the mean of the ten per-category F1 scores, so every category counts equally.

IV. Architecture

Figure 1 shows the network. Each convolution block applies 3×3 filters, batch normalisation, ReLU and 2×2 max pooling:

$$h_c(i,j) = \max\Big(0,\ \sum_{u,v} w_c(u,v)\, x(i+u, j+v) + b_c\Big),$$

and pooling takes the maximum over each 2×2 patch, halving the width and height ($28 \to 14 \to 7$). Weight sharing reuses each filter at every spatial position. A filter has nine weights per input channel, plus one bias. A 3×3 filter sees a small neighbourhood; after two blocks each unit depends on a larger part of the picture, which lets deeper blocks combine edges into shapes such as sleeves or collars.

The $64 \times 7 \times 7 = 3136$ values pass through dense layers of 128 and 64 units (ReLU, dropout between them) to a linear layer with ten outputs. The network has 429,450 trainable weights. We chose two blocks because pictures are only 28×28; after two poolings they are 7×7, and a third pooling would leave 3×3. The ablation tests a third block.

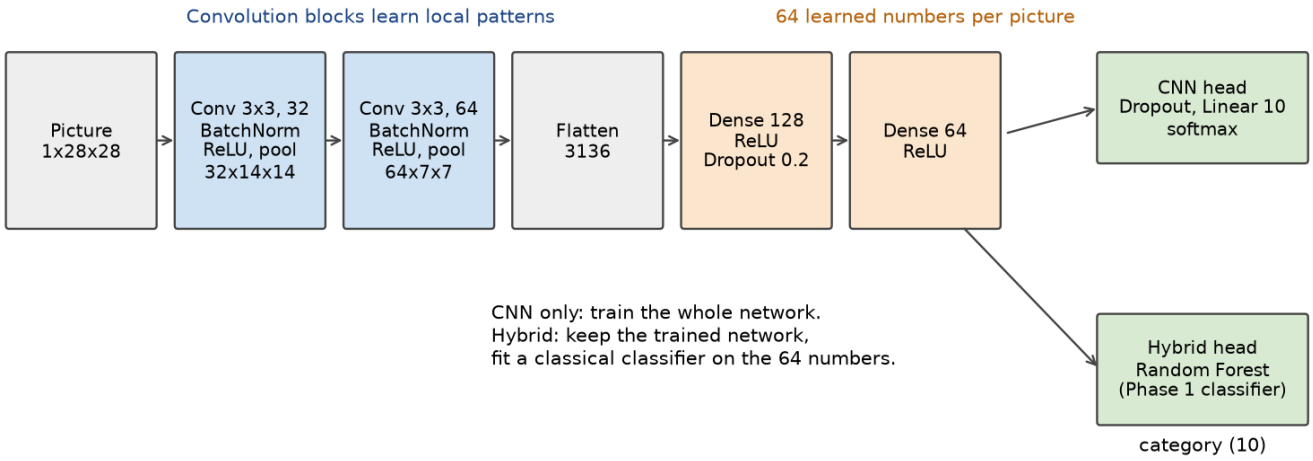


Fig. 1. The network and the two ways to finish. The 64 numbers after the dense layers are the learned features.

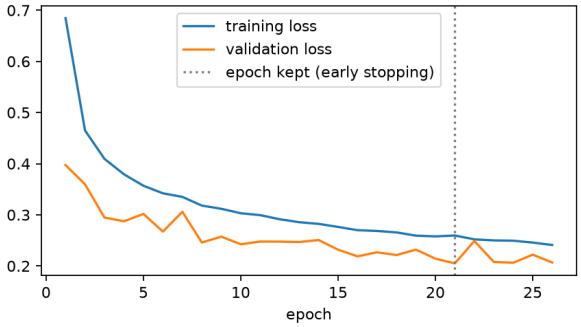


Fig. 2. Loss per epoch, final CNN, seed 42.

V. Regularisation and Training

Batch normalisation rescales each channel in a batch, $\hat{x} = (x - \mu_B)/\sqrt{\sigma_B^2 + \epsilon}$, then applies a learned scale and shift. It uses batch statistics in training and stored averages in evaluation. **Dropout** sets each unit to zero with probability p during training and rescales the rest by $1/(1-p)$; all units are used at test time. **Augmentation** mirrors each training picture left to right with probability 0.5 and shifts it by up to two pixels in each direction, filling with black. A mirrored or nudged garment keeps its category. Test pictures are never altered. **Weight decay** of 10^{-4} adds an L2 penalty through the Adam optimiser.

The loss is cross-entropy, $-\frac{1}{n}\sum_i \log p_{y_i}(x_i)$, minimised with Adam [5] (learning rate 10^{-3}, batches of 128, at most 30 epochs). **Early stopping:** after each epoch we compute validation loss; training stops after five epochs without improvement, and the weights of the best epoch are restored. Reported scores never come from a later, more overfit epoch. Figure 2 shows the final model: training loss keeps falling while validation loss flattens.

TABLE I
Ablation, validation macro F1 (mean ± std of 3 seeds).

Version	Macro F1
No dropout	0.9282 ± 0.0022
Light dropout 0.2 (final)	0.9264 ± 0.0033
Plain	0.9258 ± 0.0023
No augmentation	0.9254 ± 0.0033
Three blocks	0.9215 ± 0.0049
Full (dropout 0.4)	0.9202 ± 0.0036
No batch norm	0.9195 ± 0.0018

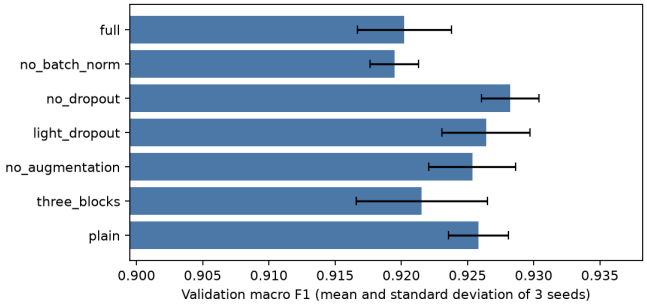


Fig. 3. Ablation. Bars show the mean, whiskers one standard deviation.

VI. Ablation Study

We remove one ingredient at a time from a full model (batch normalisation, dropout 0.4, augmentation, two blocks), add a three-block version, and run a plain model with none of the three ingredients. Light dropout (0.2) was added after the first runs showed that 0.4 hurt; it was judged on validation data only. Table I and Fig. 3 give validation macro F1.

All seven versions lie within 0.9 points, and most gaps are about as large as the seed-to-seed spread, so we rank them loosely. Dropout of 0.4 reduced macro F1 by 0.8 points compared with no dropout; dropout of 0.2 is close to no dropout. Our reading is that at this size the network overfits

TABLE II
OFFICIAL TEST SET, SCORED ONCE.

Model	Accuracy	Macro F1
Phase 1: Pixels + RF	87.61%	0.8747
CNN (3 seeds)	$91.98 \pm 0.24\%$	0.9194 ± 0.0032
Hybrid, CNN + RF (3 seeds)	$92.49 \pm 0.19\%$	0.9246 ± 0.0020

little, so strong regularisation costs accuracy; we did not test this further. Augmentation and batch normalisation showed no clear gain here, and a third block did not help. The project brief asks for a CNN with dropout, so the final model is the best version that uses it (light dropout, 0.9264). The best version overall, no dropout, is 0.0018 higher, which is smaller than one standard deviation.

VII. Hybrid Approach

The CNN's second dense layer gives a vector $z \in \mathbb{R}^{64}$ per picture. Instead of the final linear layer we fit a Phase 1 classifier on z from the training pictures: Logistic Regression, linear SVM or Random Forest, each with the two Phase 1 settings, chosen by validation macro F1. The reason is that the final layer is one linear map, while a forest or a margin classifier may draw a better boundary around the same features. The classifier is fitted after the CNN is trained, not jointly.

Two checks keep the claim honest. First, the CNN's own last layer is the comparison. Second, a control uses features from an untrained CNN of the same shape. On validation data the selected Random Forest reaches macro F1 0.9324, against 0.9264 for the CNN's last layer and 0.7944 for the untrained-network control. The control shows that training, not only the architecture, makes the features useful. The three classifier families land within 0.15 points of each other, and even Logistic Regression gains about half a point, so the gain comes from refitting on the learned features rather than from the forest itself.

VIII. Results

Table II gives test results. The CNN and the hybrid use three seeds each. A paired bootstrap over the 10,000 test pictures (2,000 resamples, seed 42 models) gives an accuracy difference of $+4.10$ points for the CNN over the Phase 1 best, 95% interval $[+3.52, +4.67]$, and $+0.60$ points for the hybrid over the CNN, interval $[+0.28, +0.94]$. The three-seed means differ by 0.51 points.

Similar categories. Figure 4 and Table III compare recall. The CNN removes most Pullover–Coat confusion (116 to 33 pullovers called coats) and raises Shirt recall by 14 points, yet about one shirt in four is still wrong. Figure 5 shows pictures the CNN gets right and Phase 1 gets wrong, and pictures both get wrong; several of the latter are long-sleeved tops with unusual patterns or shapes. The pictures alone cannot show whether a label is wrong.

Cost. On the same CPU with two threads, from picture to label, the CNN takes 0.17 ms per picture and 429,450 weights

TABLE III
TEST RECALL (CNN AND HYBRID: MEAN OF 3 SEEDS).

Category	Phase 1	CNN	Hybrid
T-shirt/top	85.2	88.9	88.5
Pullover	80.2	88.7	89.6
Coat	82.2	87.0	88.2
Shirt	59.2	73.6	75.7

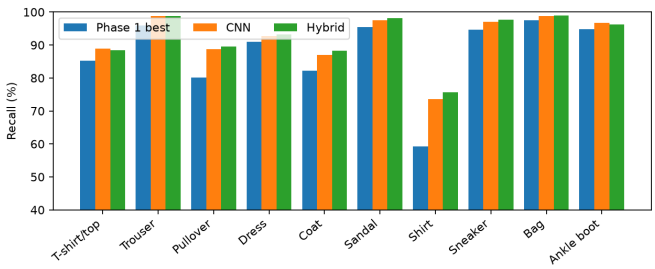


Fig. 4. Per-category test recall.

(1.7 MB). The Phase 1 forest took 24.98 ms (measured in an earlier session on the same machine). The hybrid takes 25.88 ms because its 400-tree forest dominates.

IX. Technical Validation and Limitations

Tests recompute every reported score from the saved predictions, check the committed checkpoints against the saved hashes, check that the frozen choices match the selection files, and check that the Phase 2 split equals Phase 1. All three CNN checkpoints reproduce the saved test predictions on the CPU. Rebuilding the frozen hybrid on CPU features agrees with 99.94–99.98% of the recorded predictions; the original GPU-feature results remain the reported experiment. Training on an Apple GPU is not exactly repeatable, so a rerun can differ slightly.

Limits: Fashion-MNIST is a small, centred, familiar benchmark, and these results say little about real catalogue photos. We use one split, three seeds and a small search. In the hybrid, the classifier is fitted on training-picture features from a network that already saw those pictures, which may make them look cleaner than features of new pictures. Light dropout and the rule "best version that uses dropout" were added after seeing validation results; the test pictures played no part in either. We did not test residual networks, pretrained models, schedules or other augmentations because they add machinery beyond the problem.

X. Conclusion

A two-block CNN with batch normalisation, light dropout, augmentation and early stopping beat the best Phase 1 model by about four points and helped most with Pullover, Coat and Shirt. Feeding its 64 learned features to a Random Forest added about half a point more. Most ablation choices mattered less than the seed-to-seed spread, and heavy dropout hurt. The saved CNN and an exported hybrid support verified image

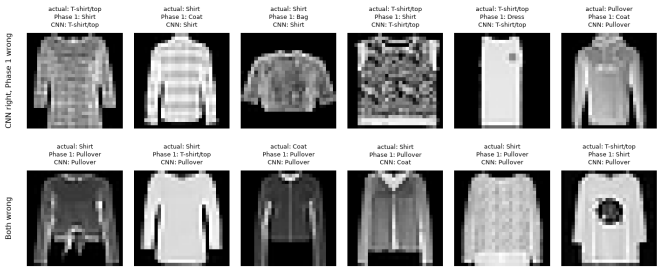


Fig. 5. Upper-body test pictures: CNN right and Phase 1 wrong (top); both wrong (bottom).

prediction in the local dashboard and command line. The hosted main-branch dashboard remains Phase 1.

REFERENCES

[1] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proc. IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.

[2] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet classification with deep convolutional neural networks," in *Advances in Neural Information Processing Systems 25*, 2012.

[3] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: a simple way to prevent neural networks from overfitting," *J. Mach. Learn. Res.*, vol. 15, pp. 1929–1958, 2014.

[4] S. Ioffe and C. Szegedy, "Batch normalization: accelerating deep network training by reducing internal covariate shift," in *Proc. ICML*, 2015, arXiv:1502.03167.

[5] D. P. Kingma and J. Ba, "Adam: a method for stochastic optimization," in *Proc. ICLR*, 2015, arXiv:1412.6980.

[6] L. Prechelt, "Early stopping—but when?" in *Neural Networks: Tricks of the Trade*, LNCS 1524, Springer, 1998.

[7] P. Y. Simard, D. Steinkraus, and J. C. Platt, "Best practices for convolutional neural networks applied to visual document analysis," in *Proc. ICDAR*, 2003, pp. 958–962.

[8] A. S. Razavian, H. Azizpour, J. Sullivan, and S. Carlsson, "CNN features off-the-shelf: an astounding baseline for recognition," in *CVPR Workshops*, 2014, arXiv:1403.6382.

[9] Y. Tang, "Deep learning using linear support vector machines," arXiv:1306.0239, 2013.

[10] H. Xiao, K. Rasul, and R. Vollgraf, "Fashion-MNIST: a novel image dataset for benchmarking machine learning algorithms," arXiv:1708.07747, 2017.